

Question No:10

Problem Description

Football is Monk's favourite sport, and his favourite team is "Manchester United." Manchester United has qualified for the Champions League Final, which will take place at London's Wembley Stadium. As a result, he decided to go watch his favourite team play.

When he arrived at the stadium, he noticed that there was a long wait for match tickets. He is aware that the stadium has M rows, each with a distinct seating capacity. They could or might not be comparable. The cost of a ticket is determined by the row. If there are K (always higher than 0) empty seats in a row, the ticket will cost K pounds (units of British Currency).

Now, every football fan standing in the line will get a ticket one by one.

Given the seating capacities of different rows, find the maximum possible pounds that the club will gain with the help of the ticket sales.

Constraints:

$1 \leq M \leq 1000000$

$1 \leq N \leq 1000000$

$1 \leq X[i] \leq 1000000$

Sum of $X[i]$ for all $1 \leq i \leq M$ will always be greater than N .

Input:

The first line consists of M and N . M denotes the number of seating rows in the stadium and N denotes the number of football fans waiting in the line to get a ticket for the match.

Next line consists of M space separated integers $X[1], X[2], X[3], \dots, X[M]$ where $X[i]$ denotes the number of empty seats initially in the i^{th} row.

Output:

Print in a single line the maximum pounds the club will gain.

Logical Test Cases

Test Case 1

INPUT (STDIN)

3 4
1 2 4

EXPECTED OUTPUT

11

Test Case 2

INPUT (STDIN)

8 4
3 7 1 2 4 7 5 9

EXPECTED OUTPUT

31

Code:

```
1  #include <stdio.h>
2
3  #define MAXM 1000000
4
5  long long heap[MAXM];
6  int size = 0;
7
8  // Insert an element into max heap
9  void insert(long long value)
10 {
11     int i = size;
12     heap[size] = value;
13     size++;
14
15     while (i > 0)
16     {
17         int parent = (i - 1) / 2;
18
19         if (heap[parent] >= heap[i])
20             break;
21
22         long long temp = heap[parent];
23         heap[parent] = heap[i];
24         heap[i] = temp;
25
26         i = parent;
27     }
28 }
```

```
30 // Remove and return maximum element
31 long long removeMax()
32 {
33     long long max = heap[0];
34
35     size--;
36
37     if (size > 0)
38     {
39         heap[0] = heap[size];
40
41         int i = 0;
42
43         while (1)
44         {
45             int left = 2 * i + 1;
46             int right = 2 * i + 2;
47             int largest = i;
48
49             if (left < size && heap[left] > heap[largest])
50                 largest = left;
51
52             if (right < size && heap[right] > heap[largest])
53                 largest = right;
54
55             if (largest == i)
56                 break;
```

```

57
58         long long temp = heap[i];
59         heap[i] = heap[largest];
60         heap[largest] = temp;
61
62         i = largest;
63     }
64 }
65
66     return max;
67 }
68
69 int main()
70 {
71     int M, N;
72
73     scanf("%d %d", &M, &N);
74
75     for (int i = 0; i < M; i++)
76     {
77         long long x;
78         scanf("%lld", &x);
79
80         insert(x);
81     }
82
83     long long revenue = 0;
84
85     for (int i = 0; i < N; i++)
86     {
87         long long seats = removeMax();
88
89         revenue += seats;
90
91         // One seat is occupied
92         if (seats > 1)
93             insert(seats - 1);
94     }
95
96     printf("%lld\n", revenue);
97
98     return 0;
99 }

```

Output:

Test Case-1:

3 4
1 2 4
11
DS (C)

Test Case-2:

```
8 4
3 7 1 2 4 7 5 9
31
```

Keshav Kumar Rauniyar
CH.SC.U4CSE25236